Práctica 3: ocho clases y abstracción

Sistema académico | Anahí Robayo

Ejemplo adaptado con las ocho clases principales de tu práctica 3: Persona, Docente, Estudiante, Asignatura, Curso, Aula, Horario e Inscripcion. Persona pasa a ser abstracta; Docente y Estudiante heredan de ella. Se conservan los datos principales y se añaden métodos para consultar los objetos.

Caso de uso: consultar una inscripción

Objetivo: registrar la inscripción de Anahí en Programación orientada a objetos y consultar sus datos académicos.

Participante: una persona encargada del registro académico ejecuta el programa. Para este ejercicio, los datos se escriben directamente en el código.

Flujo: se crean docente, estudiante, asignatura, curso, aula y horario. Después se crea una inscripción que guarda referencias a esos objetos y se llama a mostrar() para consultar la información. La calificación y asistencia del ejemplo corresponden a un registro ya evaluado.

Uso de la abstracción: al recorrer docente y estudiante, se llama a realizar_actividad(). Cada clase implementa su propia respuesta. Persona exige ese método, pero no se crea un objeto de Persona directamente.

Relación de herencia

```
Persona (ABC)
    |-- Docente
    |-- Estudiante
```

Las otras cinco clases son concretas. Inscripcion relaciona objetos mediante sus atributos; no hereda de ellos. Las ocho clases incluyen a Persona: no son ocho clases más una abstracta.

El código completo aparece a continuación, en orden. Cada parte continúa en el mismo archivo Python. No se incluyen las clases adicionales LaboratorioTI y Asistente.

Código completo | Parte 1 de 4

```
from abc import ABC, abstractmethod


# 1. Persona: clase abstracta.
class Persona(ABC):
    especie = "Humano"

    def __init__(self, nombre, identificacion):
        self.nombre = nombre
        self.identificacion = identificacion

    def mostrar(self):
        print(f"Nombre: {self.nombre} | ID: {self.identificacion}")

    @abstractmethod
    def realizar_actividad(self):
        pass


# 2. Docente: implementa el metodo abstracto.
class Docente(Persona):
    status = "En proceso"

    def __init__(self, nombre, identificacion, disponibilidad):
        super().__init__(nombre, identificacion)
        self.disponibilidad = disponibilidad

    def realizar_actividad(self):
        print(f"El docente {self.nombre} imparte la clase.")


# 3. Estudiante: implementa el mismo metodo de otra manera.
class Estudiante(Persona):
    estado = "Activo"

    def __init__(self, nombre, identificacion, matricula):
        super().__init__(nombre, identificacion)
        self.matricula = matricula

    def realizar_actividad(self):
        print(f"La estudiante {self.nombre} participa en la clase.")
```

Código completo | Parte 2 de 4

```
# 4. Asignatura.
class Asignatura:
    tipo = "Academico"

    def __init__(self, codigo, nombre, horas_semanales):
        self.codigo = codigo
        self.nombre = nombre
        self.horas_semanales = horas_semanales

    def mostrar(self):
        print(f"Asignatura: {self.nombre} | Codigo: {self.codigo}")
        print(f"Horas semanales: {self.horas_semanales}")


# 5. Curso.
class Curso:
    nivel = "3 nivel"

    def __init__(self, nombre, paralelo, semestre):
        self.nombre = nombre
        self.paralelo = paralelo
        self.semestre = semestre

    def mostrar(self):
        print(f"Curso: {self.nombre} | Paralelo: {self.paralelo}")
        print(f"Semestre: {self.semestre}")


# 6. Aula.
class Aula:
    estado_general = "Disponible"

    def __init__(self, numero, capacidad, tipo):
        self.numero = numero
        self.capacidad = capacidad
        self.tipo = tipo

    def mostrar(self):
        print(f"Aula: {self.numero} | Capacidad: {self.capacidad}")
        print(f"Tipo: {self.tipo}")
```

Código completo | Parte 3 de 4

```
# 7. Horario.
class Horario:
    modalidad = "Presencial"

    def __init__(self, dia, hora_inicio, hora_fin):
        self.dia = dia
        self.hora_inicio = hora_inicio
        self.hora_fin = hora_fin

    def mostrar(self):
        print(f"Horario: {self.dia}, {self.hora_inicio} - {self.hora_fin}")


# 8. Inscripcion: relaciona los objetos del caso de uso.
class Inscripcion:
    periodo = "2026-2027"

    def __init__(self, fecha, calificacion_final, asistencia,
                 estudiante, docente, asignatura, curso, aula, horario):
        self.fecha = fecha
        self.calificacion_final = calificacion_final
        self.asistencia = asistencia
        self.estudiante = estudiante
        self.docente = docente
        self.asignatura = asignatura
        self.curso = curso
        self.aula = aula
        self.horario = horario

    def mostrar(self):
        print(f"Fecha: {self.fecha} | Periodo: {self.periodo}")
        print(f"Estudiante: {self.estudiante.nombre}")
        print(f"Docente: {self.docente.nombre}")
        self.asignatura.mostrar()
        self.curso.mostrar()
        self.aula.mostrar()
        self.horario.mostrar()
        print(f"Calificacion final: {self.calificacion_final}")
        print(f"Asistencia: {self.asistencia}%")


# CASO DE USO: registrar y consultar una inscripcion academica.
docente1 = Docente("Carlos", "D01", "Manana")
```

Código completo | Parte 4 de 4

```
estudiante1 = Estudiante("Anahi", "E01", "M2026")
asignatura1 = Asignatura("POO01", "Programacion orientada a objetos", 4)
curso1 = Curso("Software", "A", 3)
aula1 = Aula(202, 30, "Laboratorio")
horario1 = Horario("Viernes", "08:00", "10:00")

inscripcion1 = Inscripcion(
    "16/10/2026", 9.5, 95,
    estudiante1, docente1, asignatura1, curso1, aula1, horario1
)

print("=== CONSULTA DE INSCRIPCION ===")
inscripcion1.mostrar()

print("\n=== POLIMORFISMO ===")
personas = [docente1, estudiante1]
for persona in personas:
    persona.mostrar()
    persona.realizar_actividad()

# No se crea Persona directamente porque es abstracta:
# persona1 = Persona("Ana", "P01")  # Produciria TypeError.
```

Resultado y explicación del caso de uso

```
=== CONSULTA DE INSCRIPCION ===
Fecha: 16/10/2026 | Periodo: 2026-2027
Estudiante: Anahi
Docente: Carlos
Asignatura: Programacion orientada a objetos | Codigo: POO01
Horas semanales: 4
Curso: Software | Paralelo: A
Semestre: 3
Aula: 202 | Capacidad: 30
Tipo: Laboratorio
Horario: Viernes, 08:00 - 10:00
Calificacion final: 9.5
Asistencia: 95%

=== POLIMORFISMO ===
Nombre: Carlos | ID: D01
El docente Carlos imparte la clase.
Nombre: Anahi | ID: E01
La estudiante Anahi participa en la clase.
```

¿Dónde está la clase abstracta? En Persona(ABC), con el método decorado con @abstractmethod.

¿Dónde se implementa? En realizar_actividad() de Docente y Estudiante. Si una de estas clases no lo implementa, seguirá siendo abstracta y no podrás crear sus objetos.

¿Dónde está el polimorfismo? En el bucle final: la misma llamada persona.realizar_actividad() ejecuta una respuesta diferente según el objeto.

¿Qué hace super()? Reutiliza el constructor de Persona para inicializar nombre e identificación en sus clases hijas.